

Open in app ↗

Sign up

Sign in

Medium

Search

Write



Setting up AWS S3 (Minio) and Spark using Docker Compose



Dimitris Kalouris

Follow

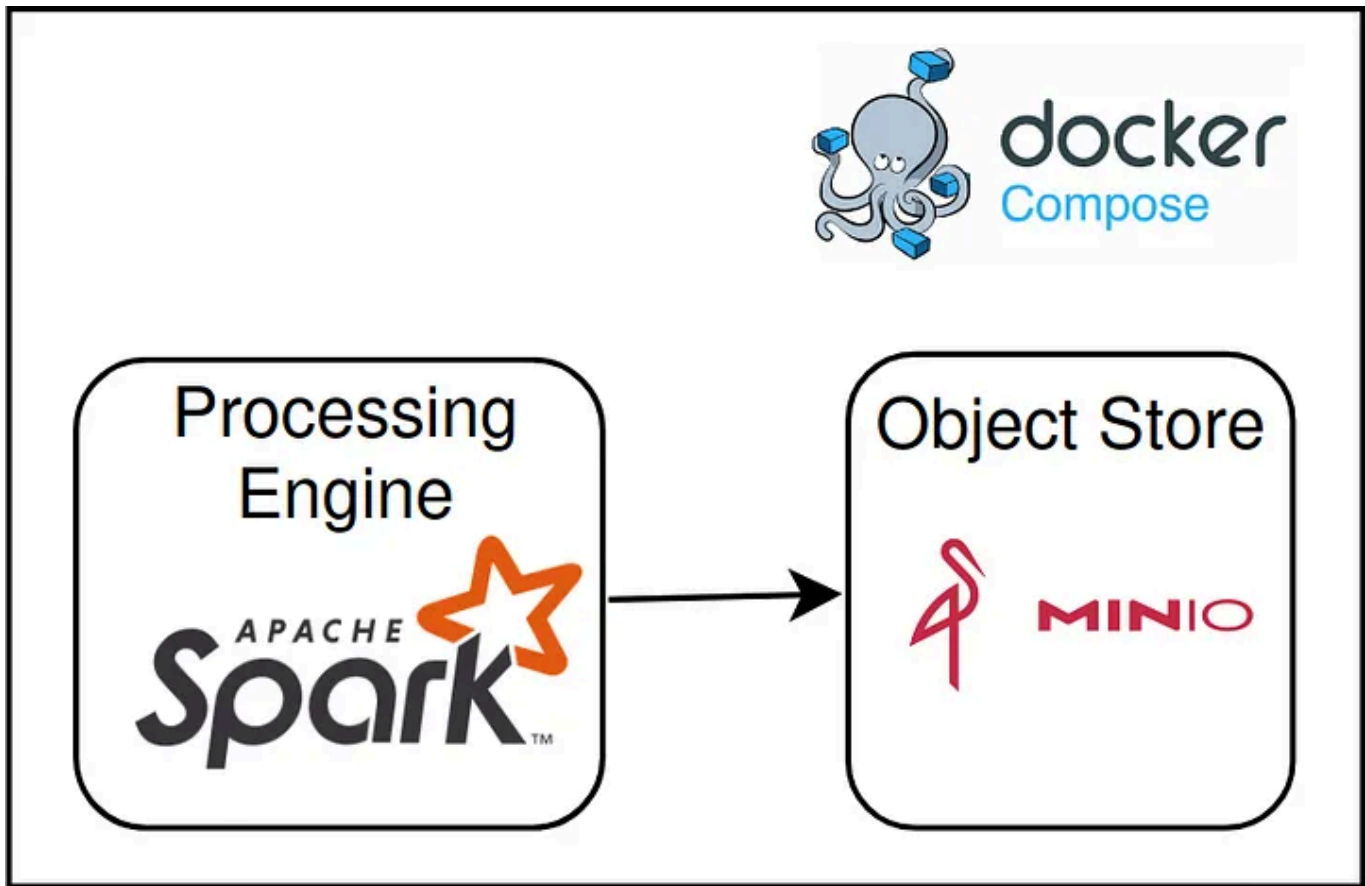
5 min read · Mar 19, 2025



53



1



Spark + MinIO

Note: This step by step guide assumes you already have a working Spark setup. If you don't check out [Part 1: Setting up Spark using Docker](#) for a quick tutorial on how to do it with docker-compose. Also if you want to run the full examples found in git see also [Part 2: Setting up a relational database \(PostgreSQL\) and Spark using Docker Compose](#).

Connecting S3 object storage to Spark can be quite challenging especially when trying to find the correct versions of dependencies to use. While in this article we will focus on **MinIO** which is an S3-compatible object storage system and **open-source alternative** to Amazon S3 (which makes it ideal for testing code locally), similar steps can be taken to connect Spark with AWS.

Table of contents:

- [Downloading Spark dependencies for AWS](#)
- [Adding MinIO](#)
- [Adding MinIO Client](#)
- [Adding connection configs to spark](#)
- [Bringing it all together and submitting Spark Scala job](#)

1. Downloading Spark dependencies for AWS

In order for Spark to use MinIO you need to add the hadoop-aws and aws-java-sdk-bundle inside spark/jars directory. In this step it is crucial to find the exact same version that your Spark setup is built with and use it to find the versions of the jars.

Fist navigate to spark/jars directory and look for hadoop references:

```

/opt/spark/jars# ls -l *hadoop*
-rw-r--r-- 1 1000 1000 15935 Feb 23 20:45 dropwizard-metrics-hadoop-metrics2-reporter-0.1.2.jar
-rw-r--r-- 1 root root 962685 Jul 29 2022 hadoop-aws-3.3.4.jar
-rw-r--r-- 1 1000 1000 19458635 Feb 23 20:45 hadoop-client-api-3.3.4.jar
-rw-r--r-- 1 1000 1000 30085504 Feb 23 20:45 hadoop-client-runtime-3.3.4.jar
-rw-r--r-- 1 1000 1000 3362359 Feb 23 20:45 hadoop-shaded-guava-1.1.1.jar
-rw-r--r-- 1 1000 1000 56812 Feb 23 20:45 hadoop-yarn-server-web-proxy-3.3.4.jar
-rw-r--r-- 1 1000 1000 1004811 Feb 23 20:45 parquet-hadoop-1.13.1.jar

```

Looking at list above we will need to download hadoop-aws-3.3.4.jar (in your case it won't be part of the list). To do this navigate to its [maven repository](#) and get the jar download link.

Then to find the correct aws-java-sdk-bundle version, in the same maven link above navigate to “compiled dependencies” and you will find it 1.12.262:

Compile Dependencies (1)

Category/License	Group / Artifact	Version	Updates
Apache 2.0	com.amazonaws:aws-java-sdk-bundle	1.12.262	2.31.1

Move to this [maven repository](#) and get one jar. And then we add these to Spark Dockerfile (or download in case of local setup).

Dockerfile:

```

# Download aws jar and add it to spark jars
RUN wget -P ${SPARK_HOME}/jars/ https://repo1.maven.org/maven2/org/apache/hadoop
RUN wget -P ${SPARK_HOME}/jars/ https://repo1.maven.org/maven2/com/amazonaws/aws

```

2. Adding MinIO

Update docker-compose with following service:

docker-compose.yml:

```
minio:
  image: quay.io/minio/minio
  command: server /data --console-address ":9001"
  container_name: minio
  networks:
    - sparkpg_network
  environment:
    - MINIO_ROOT_USER=minioadmin
    - MINIO_ROOT_PASSWORD=minioadmin
  # volumes:
    # - minio_data:/data
  ports:
    - 9000:9000
    - 9001:9001
```

This will do the following:

- Pull quay.io minio image from Docker.
- Start minio service using /data for storage and port 9001 for web UI
- Set container name to minio
- Connect it to our existing network
- Define MINIO_ROOT_USER, MINIO_ROOT_PASSWORD credentials
- Expose port 9000 for minio api .
- Expose port 9001 for web UI.

If you want minio data to be persisted even when you remove the container uncomment the volume argument.

3. Adding MinIO Client

MinIO Client (mc in short) is a command-line tool used to interact with MinIO object storage. This service will be used to add a custom policy and data to MinIO using one-shot process described in `init-minio.sh`.

First for the policy which will give the required permissions to read and write data with Spark for bucket `bucket1` :

Get Dimitris Kalouris's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

policy.json:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:PutObject",
        "s3:GetObject",
        "s3:ListBucket",
        "s3:DeleteObject"
      ],
      "Resource": [
        "arn:aws:s3:::bucket1",
        "arn:aws:s3:::bucket1/*"
      ]
    }
  ]
}
```

Note: Here DeleteObject is important as Spark generates and deletes temporary files during execution, and if this permission is missing it will create the temporary files and fail in the delete step before the final output is generated.

For sample data:

```
name,gender,age
jake,male,21
mike,male,30
jolyn,female,20
```

Next is the init-minio.sh shell script used to combine those two:

```
#!/bin/sh

# Wait for Minio to start and add minio admin alias
until mc alias set myminio http://minio:9000 minioadmin minioadmin; do
    echo "MinIO is not ready yet. Retrying in 5 seconds..."
    sleep 5
done

# Make bucket and add data
mc mb myminio/bucket1
mc cp ./sample.csv myminio/bucket1/sample.csv

# Add new user and attach new policy
mc admin user add myminio testuser password
mc admin policy create myminio read-write policy.json
mc admin policy attach myminio read-write --user testuser
```

This will do the following:

- Set alias for MinIO url using admin credentials provided there and retry every 5 seconds until it succeeds.
- Make new bucket called bucket1
- Upload `sample.csv` data to `bucket1/sample.csv`
- Add new user testuser
- Add new policy called read-write using file `policy.json`
- Attach new read-write policy to testuser

Finally Update docker-compose with following service:

docker-compose.yaml:

```
mc:
  image: quay.io/minio/mc
  container_name: mc
  depends_on:
    - minio
  networks:
    - sparkpg_network
  volumes:
    - ./init-minio.sh:/init-minio.sh
    - ./minio_data.csv:/sample.csv
    - ./policy.json:/policy.json
  entrypoint: ./init-minio.sh
```

This will do the following:

- Pull quay.io mc image from Docker.
- Connect it to our existing network

- Mount required files to mc
- Execute init-minio.sh

4. Adding connection configs to spark

To execute this with spark we need to point it to the url of MinIO and add the credentials needed to access it. To do this update spark-defaults with the following:

spark-defaults.conf:

```
# Set aws parameters
# For local setup use localhost instead of minio in endpoint
spark.hadoop.fs.s3a.endpoint http://minio:9000
spark.hadoop.fs.s3a.impl org.apache.hadoop.fs.s3a.S3AFileSystem
spark.hadoop.fs.s3a.access.key testuser
spark.hadoop.fs.s3a.secret.key password
spark.hadoop.fs.s3a.path.style.access true
```

Note: If you instead add these commands to spark code like `spark.conf.set("config_name","value")` the code will fail! This is because they need to be provided before the spark context is created and they cannot be changed afterwards!

5. Bringing it all together and submitting Spark Scala job

To test that policy works we will try to read the csv data file found at `s3a://bucket1/sample.csv` then convert it to parquet format and write back to MinIO.

spark code:


```
import org.apache.spark.sql.{DataFrame, SparkSession}
object SparkS3Example {
  def main(args: Array[String]): Unit = {
    val spark: SparkSession = SparkSession.builder
      .appName("sparkS3")
      .getOrCreate()

    // Read data and print
    val data : DataFrame = spark.read.option("header","true").csv("s3a://bucket1")
    data.write.mode("overwrite").parquet("s3a://bucket1/sample_parquet")
  }
}
```

To confirm the execution we will look inside `bucket1` before and after execution by connecting to MinIO container:

```
$docker exec -it minio /bin/bash
$mc alias set myminio http://minio:9000 minioadmin minioadmin
$mc ls myminio/bucket1
```

before:

```
mc ls myminio/bucket1
57B STANDARD sample.csv
```

after:

```
mc ls myminio/bucket1
57B STANDARD sample.csv
0B sample_parquet/
mc ls myminio/bucket1/sample_parquet
0B STANDARD _SUCCESS
971B STANDARD part-00000-feaec3fe-3cfd-4304-8860-84681458d521-c000.snappy.parquet
```

Thanks for reading!

All the code and more examples including reading from PostgreSQL and writing to MinIO are available on Github [here](#).

This is Part 3 of an end to end project that will feature Spark, PostgreSQL, Minio and Airflow using Docker Compose.

Next, check out [Part 4: Adding scheduling using Airflow](#).

Spark

S3

Minio

Docker

AWS

**Written by Dimitris Kalouris**

28 followers · 8 following

Follow

Data Practitioner with passion for machine learning, distributed computing, and working at all stages of the data life-cycle.

Responses (1)



Write a response

What are your thoughts?



Alessandro Peretti

Aug 5, 2025



There is a mistake in bullet point list:

port 9001 = web UI

port 9000 = API



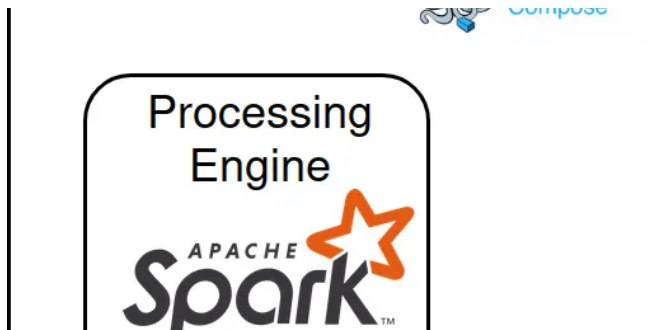
1



1 reply

[Reply](#)

More from Dimitris Kalouris



Dimitris Kalouris

Setting up Spark using Docker

Quick tutorial on how to create Dockerfile and the Docker Compose services needed to run...

Feb 3, 2025



65



2



Dimitris Kalouris

Setting up and connecting Airflow and Spark using Docker Compose

Step by step guide on how to setup and connect Airflow with Spark and execute DAG...

Apr 27, 2025

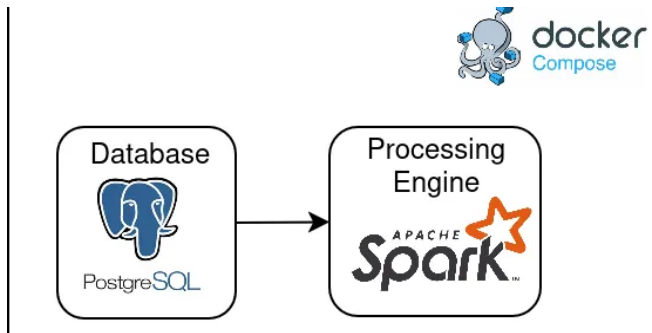


54



2





 Dimitris Kalouris

Setting up a relational database (PostgreSQL) and Spark using...

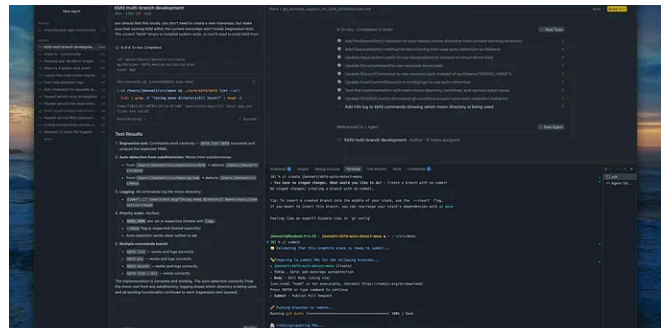
Step by step guide on how to add PostgreSQL (or similarly any relational database) to an...

Feb 24, 2025  74



See all from Dimitris Kalouris

Recommended from Medium



 Ajay Kumar

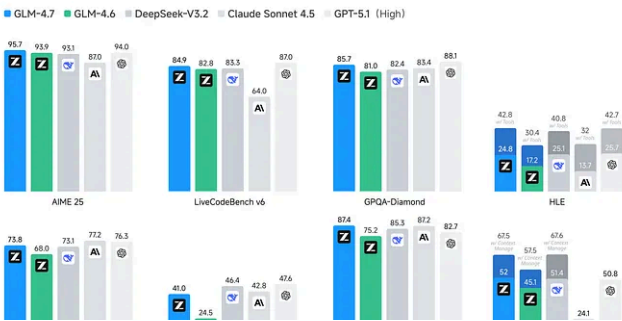
Resources, Scopes, Permissions & Policies in Keycloak — A Practical...

You've got ~250 “capabilities” today (e.g., viewAccount, viewTransaction) granted via...

★ Oct 18, 2025

👤 3

💬



 Agent Native

Local LLMs That Can Replace Claude Code

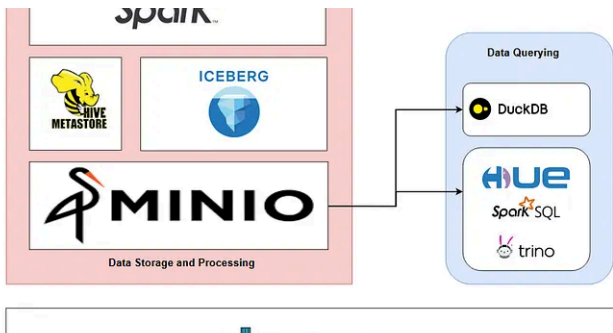
Editor's note: While hardware tiers remain valuable context for this article, we've...

★ Jan 20

👤 1.8K

💬 52

💬



 Kavinda Thennakoon

 Jacob Bennett

The 5 paid subscriptions I actually use in 2026 as a Staff Software...

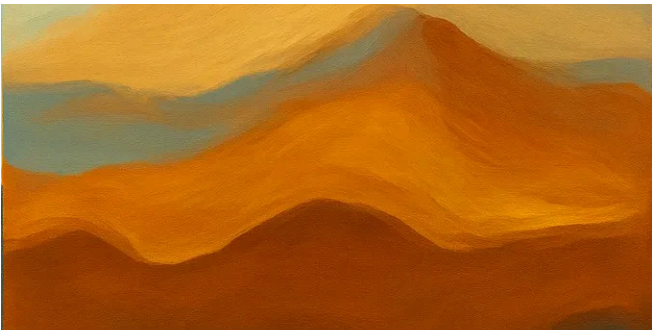
Tools I use that are (usually) cheaper than Netflix

★ Jan 19

👤 4.4K

💬 103

💬



 Engineer Palsu

Initializing and Managing a Docker Swarm Cluster

Initializing a Docker Swarm cluster is the foundational step in creating a highly...

★ Dec 16, 2025

👤 51

💬

	S3 Tables	DIY Iceberg	Difference
7yr avg)	\$19,320	\$6,216	-\$13,104
Cost	Included	\$168 (7yr)	+\$168
Time	0 hours	21 hours (7yr)	+21 hours
Performance	High	Low	✓
Reliability	Unknown	Proven	✓
Flexibility	AWS-only	Any platform	✓
Cost	\$19,320	\$6,384	-\$12,936

 In AWS in Plain English by Heinan Cabouly

How I Brought Iceberg, Spark, HUE, Trino, and DuckDB to My MINIO...

In my first article, I shared a detailed, step-by-step guide on how to build an on-premises...

Why I Rejected AWS's New S3 Tables for FDA Compliance (And...

Picture this: It's 2 AM. Your FDA audit starts in 48 hours. The auditor requests audit logs...

Nov 7, 2025



1



Oct 26, 2025



8



See more recommendations